

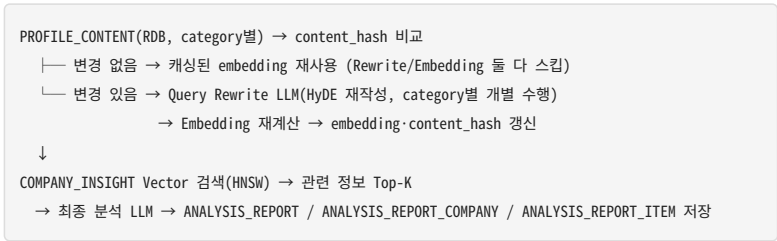
AI-Ready 설계 정리 v2 (ERD 반영)

1. AI가 들어가는 위치

구분	역할	분류
맞춤 기업 추천 및 분석	사용자 Profile과 여러 기업 정보를 비교해 추천 기업(들)과 준비 방향 제공	AI 확장 지점
특정 기업 적합도 분석	사용자 Profile과 목표 기업 정보를 비교해 Fit 분석 결과 제공	AI 확장 지점
Query Rewrite	PROFILE_CONTENT를 COMPANY_INSIGHT와 같은 장르의 문서로 재작성	내부 AI 처리
Embedding / Vector Search	재작성된 문서를 벡터화해 COMPANY_INSIGHT와 유사도 검색	내부 기술 처리

PDF 생성은 AI 기능이 아니라 분석 결과를 하나의 Report로 묶어 제공하는 일반 기능으로 본다.

2. 전체 처리 흐름



해시 비교가 Query Rewrite보다 항상 먼저 실행되어, 원본이 안 바뀌었으면 Rewrite(LLM 호출) 자체를 스킵한다 (자세한 이유는 3번 참고).

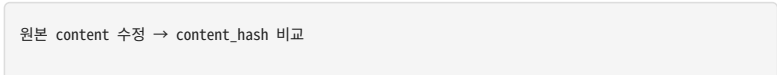
3. Query Rewrite (HyDE 방식, category별 개별 수행)

기존 논의(멀티쿼리 확장)에서 **단일 HyDE 재작성 방식으로 확정**한다. 사용자의 표현을 검색용 문장 여러 개로 쪼개는 대신, PROFILE_CONTENT의 각 category(PROJECT / EXPERIENCE / STUDY / PREFERRED_CULTURE)마다 COMPANY_INSIGHT와 같은 “장르”의 가상 문서 1개로 재작성한다. 목적은 검색어 확장이 아니라, 서로 다른 장르의 텍스트(개인 서술 vs 기업 인터뷰) 간 유사도 비교 정확도를 높이는 것이다.

구분	내용
Input	PROFILE_CONTENT.content (category 별 원문 1건)
Prompt	사용자의 경험/선호가 의미하는 바를 바꾸지 말고, 기업 인터뷰 정보(실제업무·기업문화·현업Tip 등)와 같은 형식의 가상 문서 1개를 생성
Output	{ "rewrittenContent": "..."}

적용 범위: PROFILE_CONTENT의 모든 category에 동일하게 적용한다 (선호뿐 아니라 프로젝트/경험/공부한 내용도 전부 HyDE 재작성 대상).

캐싱 전략: 기업이나 시점에 따라 프롬프트/모델을 다르게 쓰지 않으므로(요청 컨텍스트에 무관하게 동일 변환), PROFILE_CONTENT.content_hash로 원본 대비 변경 여부만 확인하면 충분하다.



→ 동일 → HyDE 재작성/임베딩 모두 스킵, 캐싱된 embedding 재사용
→ 변경 → HyDE 재작성 실행 → 결과 임베딩 → embedding, content_hash 갱신

PROFILE_CONTENT.embedding은 검색 대상(코퍼스)이 아니라 쿼리 입력값이므로 HNSW 인덱스를 걸지 않는다. 반대로 COMPANY_INSIGHT.embedding은 검색 대상이므로 HNSW 인덱스가 필수다.

4. 데이터 저장 기준

저장소	저장 데이터	이유
RDB	JOB_SEEKER_PROFILE(desired_position_id), JOB_SEEKER_TECH(tech_id, N:M) — 정확한 조건 필터링용	정형 데이터의 저장·수정·조회, WHERE 조건 매칭
RDB	PROFILE_CONTENT(프로젝트/경험/공부한 내용/선호 원문 + 캐싱된 embedding)	서술형 원문 보관 + 재계산 비용 절감용 캐싱
RDB(pgvector) + HNSW 인덱스	COMPANY_INSIGHT(현직자 인터뷰 기반 기업문화·실제업무·프로젝트경험·현업Tip)	문장 의미 기반 유사도 검색 대상(코퍼스)

별도 벡터DB 서버 없이 PostgreSQL(pgvector) 하나로 RDB와 벡터 검색을 통합 관리한다.

5. 맞춤 기업 추천 및 분석

사용자가 특정 기업을 정하지 않은 상태에서, JOB_SEEKER_PROFILE(직무/기술스택 필터) + PROFILE_CONTENT 임베딩으로 COMPANY_INSIGHT를 검색해 여러 기업 후보를 비교한다. 추천 기업 개수는 확장 가능하게 설계했으므로(ANALYSIS_REPORT_COMPANY가 1:N), 결과는 배열 형태로 여러 곳을 담을 수 있다.

구분	내용
Input	희망 직무, 기술스택(메타데이터 필터) + PROFILE_CONTENT 임베딩(HyDE 재작성본) + 검색된 여러 COMPANY_INSIGHT
Prompt	사용자와 기업 정보를 비교해 적합한 기업(들)을 추천하고 이유·강점·Gap·보완 방향을 생성. 제공된 정보만 근거로 사용
Output	기업별 reason, strengths, gaps, recommendedActions, preparationDirection

Output 형태

```
{
  "recommendations": [
    {
      "rank": 1,
      "companyName": "A기업",
      "reason": "추천 이유",
      "strengths": ["..."],
      "gaps": ["..."],
      "recommendedActions": ["..."],
      "preparationDirection": "..."
    },
    {
      "rank": 2,
      "companyName": "B기업",
      "reason": "...",
      "strengths": ["..."],
      "gaps": ["..."],
      "recommendedActions": ["..."],
      "preparationDirection": "..."
    }
  ]
}
```

저장 매핑

```
ANALYSIS_REPORT (report_type = 'RECOMMEND')
└ ANALYSIS_REPORT_COMPANY × N (rank, company_code — fit_score는 NULL)
  └ ANALYSIS_REPORT_ITEM (REASON / STRENGTH / GAP / RECOMMENDED_ACTION /
    PREPARATION_DIRECTION)
```

6. 특정 기업 적합도 분석

사용자가 목표 기업을 선택하면 해당 기업으로 검색 범위를 제한하고, JOB_SEEKER_PROFILE + PROFILE_CONTENT와 그 기업의 COMPANY_INSIGHT를 비교한다. 항상 기업 1곳에 대한 결과이므로 ANALYSIS_REPORT_COMPANY는 정확히 1행만 생성된다.

6-1. Fit Score 산정 기준

Fit Score는 LLM이 임의로 생성하지 않고, **백엔드에서 결정론적으로 계산**한 뒤 그 값과 근거를 최종 분석 LLM에 전달한다. 같은 사용자-기업 조합이면 항상 같은 점수가 나오도록 재현성을 확보하기 위함이며, LLM은 계산된 점수를 설명(fitReasons)하는 역할만 담당한다.

항목	배점	계산 방식	데이터 출처
기술스택 일치도	30점	(겹치는 기술 수 ÷ 기업 요구 기술 수) × 30	JOB_SEEKER_TECH ∩ COMPANY_INSIGHT_TECH
직무 일치도	20점	완전 일치 20점 / 불일치 0점	desired_position_id vs COMPANY_INSIGHT.position_id
프로젝트·경험 연관성	30점	벡터 유사도 평균 × 30	PROFILE_CONTENT(PROJECT/EXPERIENCE) ↔ COMPANY_INSIGHT(실제업무/프로젝트) 코사인 유사도
조직문화 적합도	20점	벡터 유사도 평균 × 20	PROFILE_CONTENT(PREFERRED_CULTURE) ↔ COMPANY_INSIGHT(기업문화/인재상) 코사인 유사도

계산 흐름

- 1. 벡터 검색 시점에 코사인 유사도 값 확보 (HNSW 검색 결과에 포함)
- 2. 백엔드에서 위 공식대로 4개 항목을 결정론적으로 계산 → fit_score 확정
- 3. fit_score + 항목별 근거 데이터를 최종 분석 LLM에 함께 전달
- 4. LLM은 점수를 생성하지 않고, 그 점수가 나온 이유(fitReasons)만 서술

확정 필요 사항 (배점·공식은 초기안이며 조정 가능하도록 설정값으로 분리):- 기업 쪽 기술 태그가 없는 인사이트일 때 기술스택 항목의 처리 방식(스킵 후 나머지 3개 항목으로 재분배 등) - 직무 불일치를 이분법(0/20점)으로 돌지, 유사 직무에 부분 점수를 줄지

구분	내용
Input	사용자 기술/프로젝트/경험 (PROFILE_CONTENT 임베딩) + targetCompany + 해당 기업 COMPANY_INSIGHT + 백엔드 계산된 fit_score
Prompt	계산된 Fit Score와 근거 데이터를 바탕으로 fitReasons를 서술하고, 강점, 부족 역량, 보완할 점, 추천 학습/프로젝트/경험, 준비 방향, 이력서 강조 Point를 생성
Output	fitScore(백엔드 계산값 그대로 반영), fitReasons, strengths, gaps, recommendedLearning, recommendedProjects, preparationDirection, resumeHighlights

Output 형태

```
{
  "companyName": "A기업",
  "fitScore": 78,
  "fitReasons": ["..."],
  "strengths": ["..."],
  "gaps": ["..."],
  "recommendedLearning": ["..."],
  "recommendedProjects": ["..."],
  "preparationDirection": "...",
  "resumeHighLights": ["..."]
}
```

저장 매핑

```
ANALYSIS_REPORT (report_type = 'FIT_ANALYSIS')
└ ANALYSIS_REPORT_COMPANY × 1 (company_code, fit_score — rank는 NULL)
  └ ANALYSIS_REPORT_ITEM (REASON / STRENGTH / GAP / RECOMMENDED_LEARNING /
RECOMMENDED_PROJECT / PREPARATION_DIRECTION / RESUME_HIGHLIGHT)
```

※ Fit Score 산정 기준은 6-1 참고. 배점 비율과 세부 처리 정책(기술스택 분모 0 처리, 직무 불일치 부분점수 여부)은 이후 실제 데이터로 검증하며 조정한다.

정리: Use-Case의 AI 확장 지점은 2개(맞춤 기업 추천 및 분석, 특정 기업 적합도 분석)로 유지한다. Query Rewrite(HyDE 재작성)는 내부 AI 처리이며, Embedding/Vector Search는 내부 기술 처리다. 두 AI 확장 지점의 결과는 ANALYSIS_REPORT → ANALYSIS_REPORT_COMPANY → ANALYSIS_REPORT_ITEM 구조로 저장되며, 추천은 기업 N개(확장 가능), 적합도분석은 기업 1개로 같은 구조를 공유한다.